Latency-Aware VLA on AMD:
Real-Time Chunking for Closed-Loop Physical AI

AMD AI DevMaster Hackathon 2026 — Track 3: Physical AI Challenge
Author: Zhang Boyu · boyuzhang1218@gmail.com · github.com/zby1218
Demo: https://youtu.be/rXQQ4s-CxEE
Frozen results: 2026-08-06

1. Target application

We build a closed-loop kitchen pick-and-place controller for a simulated Franka Panda: pick a fruit proxy and place it in a basket under the frozen instruction “Pick up the fruit and place it in the basket.” The application focus is not only training a VLA, but making large action-chunk policies usable under real AMD inference latency (≈300–350 ms/chunk) via Sync / Async / Real-Time Chunking (RTC).

2. System architecture and solution design

End-to-end loop on one AMD Radeon GPU (gfx1100, 48 GB, ROCm 7.2.1, PyTorch 2.9.1+rocm, Genesis 1.3.1, LeRobot 0.4.4):

- Data: Genesis GPU-rendered expert demos → LeRobot v3 dataset (120 episodes).
- Train: SmolVLA B0 baseline; π0.5 B1→B1d (vision unfreeze unlocks grasp).
- Execute: Sync-full E=50 (capability) vs Async/RTC E=10 under native/+200 ms delay.
- Eval: paired seed=99, GPU render only, shared smooth_action(0.15); Env-B held-out 3–5 cm fruit XY shift (never in training).

3. Datasets (train / eval)

Dataset	Role	Notes
Env-A 120 ep	Train + Env-A eval	local/franka-kitchen-fruit-place-120ep
Env-B runtime shift	Eval only	|Δxy|~U[3,5] cm; paired nominal poses

Cameras: overhead + wrist RGB 640×480 @ 30 Hz. State/action: 9-D joint positions. Generator failures (mostly not_in_basket) are skipped and not written to train data.

4. AMD Radeon GPU utilization

Stage	On Radeon?
Genesis physics + GPU render (no llvmpipe in formal scores)	Yes
Expert demo generation	Yes
SmolVLA / π0.5 BF16 training	Yes (peak VRAM 2.3→32.3 GB)
Chunk inference + RTC/Async planning	Yes
Latency-injected closed-loop eval + video	Yes (EGL)

Measured: SmolVLA train ≈6.68 step/s, peak 2.30 GB; π0.5 B1d ≈1.5 step/s, peak 32.34 GB; chunk infer P50 ≈343 ms (SmolVLA) / 305 ms (π0.5 B1d).

5. Innovations and key technical contributions

- Latency-aware Physical AI framing on AMD: measure freeze-on-replan and compare Sync / Async / RTC under injected delay on the same checkpoints and seeds.
- Primary finding: under +200 ms, Async continuous queue recovers place/lift vs Sync freeze; RTC removes freezes but does not beat Async on place (honest negative).
- π0.5 adaptation: expert-only fails closed-loop grasp; full+unfreeze vision (B1d) unlocks 1/20 place on this task.
- Env-B held-out 3–5 cm shift zeros place for both policies — pose sensitivity, not claimed as open-world generalization.

6. Final deliverables and outputs

- Source: latency_aware_vla/ in fork zby1218/Radeon-hackathon-2026-07.
- Technical reports: this PDF + paper PDF (LaTeX) + paper/ source.
- Reproducibility README + ENVIRONMENT.md + requirements-lock.txt + Dockerfile.
- Formal JSON: artifacts/formal/ (RESULTS.md + eval/train summaries).
- Demo video (3–5 min, EN): https://youtu.be/rXQQ4s-CxEE
- Figures/PDF binaries also on Release tag track3-artifacts.

7. Strengths / unique aspects

Full AMD loop (data→train→infer→eval) with frozen protocol and honest reporting of negatives (RTC ≤ Async on place; Env-B collapse). Judges can recompute tables from committed JSON rather than screenshots alone.

8. Team members and contributions

Solo submission: Zhang Boyu (boyuzhang1218@gmail.com). Responsible for Env-A data generation, SmolVLA/π0.5 training on ROCm, Sync/Async/RTC latency eval, Env-B, formal artifacts, technical report, and demo video.

9. Frozen results (seed=99)

9.1 Capability (Sync-full E=50, native)

Policy	Place	Lift	Infer P50
SmolVLA B0	2/20	3/20	343 ms
π0.5 expert-only	0/20	0/20	304 ms
π0.5 B1d	1/20	1/20	305 ms

9.2 Latency (SmolVLA)

Mode	Delay	Place	Lift	Freeze
Sync-full	0	2/20	3/20	—
Sync-full	+200 ms	0/20	2/20	yes
Async E=10	+200 ms	1/20	4/20	0
RTC E=10	+200 ms	0/20	1/20	0

9.3 Env-B (held-out 3–5 cm)

Policy	Env-A place	Env-B place
SmolVLA	2/20	0/20
π0.5 B1d	1/20	0/20

10. Reproducibility

Follow README.md: activate /workspace/rdna (or Docker recipe), set HF_LEROBOT_HOME, obtain dataset/weights (DATA_AND_WEIGHTS.md), then run scripts/run_train_*.sh and scripts/run_*_matrix.sh. Formal numbers must come from artifacts/formal/ JSON. Docker is provided as a preferable packaging recipe for ROCm PyTorch; full weights are not baked into the image.

11. Conclusion

We delivered a reproducible AMD ROCm Physical AI protocol for latency-aware VLA control. The strongest delay result is architectural continuity (Async vs Sync freeze), not an automatic RTC place boost. π0.5 required vision unfreeze to grasp. Env-B shows pose sensitivity on this small dataset. Together these define an honest latency benchmark for flow-based VLAs on AMD.